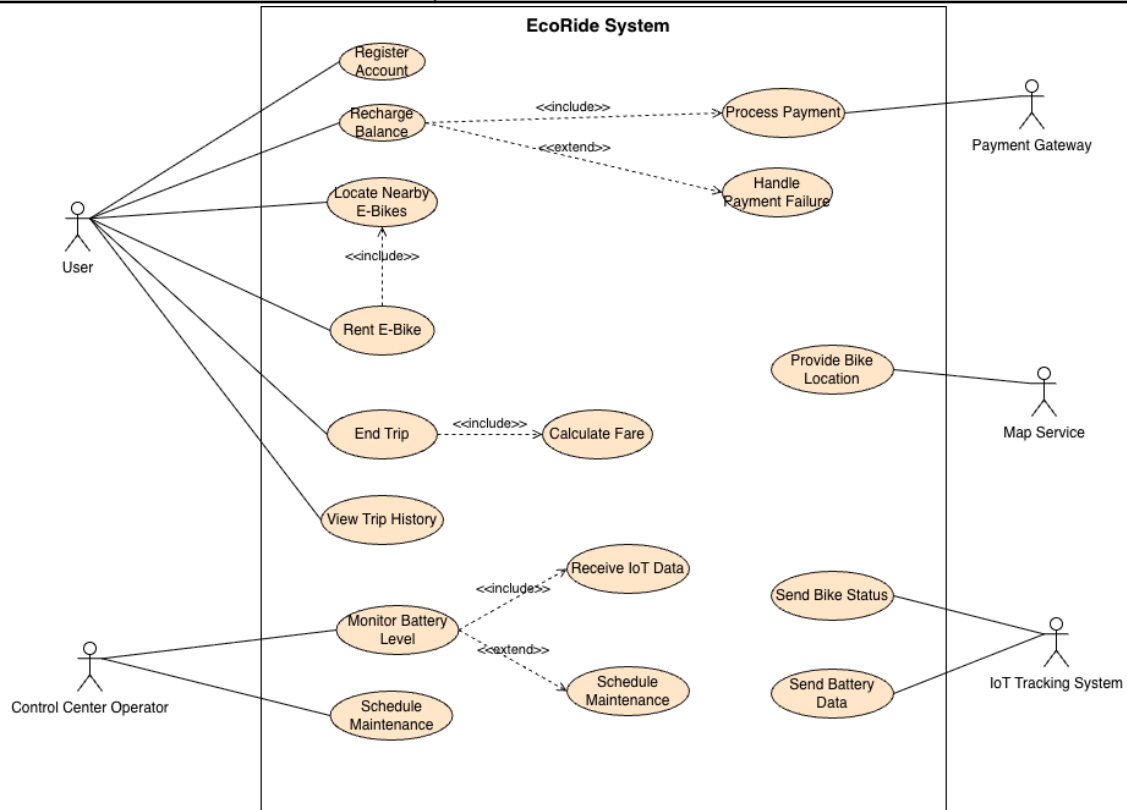


Question 1: Use Case Diagram

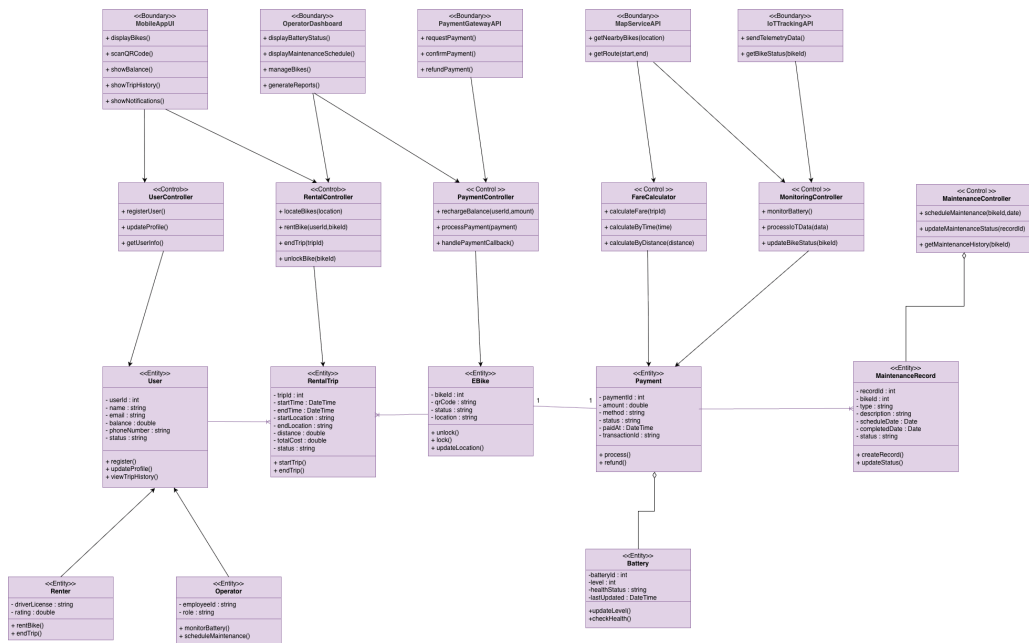
Actor	Vai trò
User (Renter)	Thuê và sử dụng e-bike
Control Center Operator	Quản lý đội xe và bảo trì
Payment Gateway	Xử lý thanh toán
Map Service	Cung cấp vị trí xe
IoT Tracking System	Cung cấp dữ liệu GPS, pin và trạng thái xe

ID	Use Case
UC1	Register Account
UC2	Recharge Balance
UC3	Locate Nearby E-Bikes
UC4	Rent E-Bike
UC5	End Trip
UC6	View Trip History
UC7	Calculate Fare
UC8	Process Payment
UC9	Monitor Battery Level

UC10	Schedule Maintenance
UC11	Receive IoT Data
UC12	Handle Payment Failure



Question 2: Class Diagram



Entity Classes

Class	Responsibility
User	Stores user information and account balance
EBike	Stores bike information, location, and status
Battery	Stores battery level and health information
RentalTrip	Stores rental trip details such as start/end time, distance, and cost
Payment	Stores payment transaction information
MaintenanceRecord	Stores maintenance schedules and maintenance history

Boundary Classes

Class	Responsibility
-------	----------------

MobileAppUI	Interface used by renters to register, recharge balance, locate bikes, rent bikes, and view trip history
OperatorDashboard	Interface used by operators to monitor bikes and manage maintenance
PaymentGatewayAPI	Interface to external payment services
MapServiceAPI	Interface to external map and location services
IoTTrackingAPI	Interface to receive telemetry data from IoT devices

Control Classes

Class	Responsibility
UserController	Handles user registration and profile management
RentalController	Handles bike searching, rental operations, and trip management
PaymentController	Handles balance recharge and payment processing
FareCalculator	Calculates rental fees based on time and distance
MonitoringController	Processes IoT data and monitors bike status
MaintenanceController	Manages maintenance scheduling and maintenance records

Question 4:

4.1 Why Service-Oriented Architecture was chosen

Service-Oriented Architecture (SOA) was selected because the EcoRide system needs to integrate with multiple external services, including payment gateways, map services, and IoT tracking systems.

SOA enables the system to be divided into independent services, each responsible for a specific business function. This separation improves

scalability, maintainability, and flexibility while allowing services to be developed, deployed, and updated independently.

In addition, SOA supports future expansion, such as adding new payment providers, map providers, or advanced bike monitoring features without affecting the entire system.

4.2 Major Subsystems / Components

The proposed architecture consists of the following major services:

1. Mobile Application (Client)

Responsibilities:

- User registration
- Account management
- Balance recharge
- Bike search
- QR code scanning
- Bike rental
- Trip history viewing

Communicates with:

- API Gateway

2. API Gateway

Responsibilities:

- Entry point for all client requests
- Request routing
- Authentication and authorization
- Service coordination

Communicates with:

- User Service
- Rental Service
- Payment Service
- Monitoring Service
- Maintenance Service

3. User Service

Responsibilities:

- User registration
- User profile management
- Wallet balance management

Data Managed:

- User

4. Rental Service

Responsibilities:

- Locate nearby bikes
- Unlock bikes
- Start trips
- End trips

Data Managed:

- EBike
- RentalTrip

Uses:

- Map Service

5. Payment Service

Responsibilities:

- Recharge balance
- Payment processing
- Transaction management

Data Managed:

- Payment

Uses:

- Payment Gateway

6. Monitoring Service

Responsibilities:

- Receive IoT data
- Monitor bike location
- Monitor battery levels

Data Managed:

- Battery
- Bike Status

Uses:

- IoT Tracking System

7. Maintenance Service

Responsibilities:

- Generate maintenance schedules
- Track maintenance records

Data Managed:

- MaintenanceRecord

8. External Systems

Payment Gateway

Provides:

- Payment processing

Map Service

Provides:

- Bike location
- Route information

IoT Tracking System

Provides:

- GPS location
- Battery level
- Bike status

4.3 Software Quality Attributes

Scalability

Each service can be scaled independently according to demand.

For example:

- Rental Service can be scaled during peak hours.
- Payment Service can be scaled during high transaction periods.

Modifiability

New features can be added without affecting the entire system.

Examples:

- Add MoMo payment gateway
- Add Google Maps alternative
- Add new bike models

Maintainability

Services are separated based on business responsibilities.

This makes debugging, testing, and updating easier.

Performance

Each service processes a specific business function, reducing system complexity and improving response time.

Caching and load balancing can be applied independently to critical services.

Security

API Gateway provides:

- Authentication
- Authorization
- Request validation

Sensitive payment operations are isolated within the Payment Service.

Availability

If one service fails, other services can continue operating.

For example:

- Payment Service failure does not affect bike monitoring.
- Monitoring Service failure does not affect user registration.

